

Primeiro o teste

O código ainda não existe. Quem diz o que ele faz é você.

Se o teste passa de primeira, ele não está testando nada.

quarta-feira, 09/09/2026 · 18h às 22h · Laboratório de Informática

Unidade Curricular 11 · Executar teste e implantação de aplicativos computacionais

NOME: _____

P A R T E

0

Se você faltou ontem

- O necessário da Aula 6, em uma página
- Você entra hoje sem dívida nenhuma

O necessário da Aula 6

O projeto de hoje é outro e não depende do de ontem. Se você faltou, precisa de três ideias — e elas cabem nesta página.

TESTE AUTOMÁTICO

Um método comum, com `[TestMethod]` em cima. Ele roda sozinho e acende verde ou vermelho.

ASSERT

A linha que decide a cor. Ela compara o que se esperava com o que o programa devolveu.

TEST EXPLORER

A janela do Visual Studio que acha os testes, roda todos e mostra o placar.



E o resto?

Ontem a turma escreveu dezoito testes contra um programa pronto que tinha seis defeitos escondidos. Nada daquilo é cobrado hoje: o programa desta noite não tem defeito nenhum, porque não tem código nenhum.

P A R T E

1

0 código que não existe

- Ontem ele mentia; hoje ele nem fala
- Assinatura, corpo, e o que falta
- Se não há código, quem decide o que é certo?

Ontem × hoje

	Aula 6	Aula 7 — hoje
O programa	estava pronto	não existe
Ele compilava e	rodava, e mentia	roda, e para
A regra estava	escrita no comentário	na sua cabeça
Os nomes dos testes	vinham prontos	saem da sua folha
Você provava que	o código estava errado	isto é o que 'certo' significa
O teste vinha	depois do programa	antes do programa



A inversão da noite

Todo o resto da unidade curricular trata o teste como quem confere um trabalho já entregue. Hoje ele é o contrário: é a encomenda.

A janela abre, e não sabe fazer nada

Aperte F5 antes de qualquer outra coisa. A janela abre inteira — os campos, os nove botões, os saldos das duas contas. Clique em Somar, e depois em Depositar:

■ Primeiro o teste - UC11 Aula 7 - 0 de 9 implementados

PRIMEIRO O TESTE
Cada botao chama um metodo. Enquanto o metodo nao tiver corpo, a resposta vai ser AINDA NAO IMPLEMENTADO.

Calculadora

Primeiro numero: Segundo numero:

Somar

Subtrair

Multiplicar

Dividir

Porcentagem

Somar AINDA NAO IMPLEMENTADO.
Escreva o teste, veja o vermelho, e so entao o corpo.

Conta bancaria

Conta A: 0 Conta B: 0

Valor:

Zerar as duas contas

Depositar

Sacar

TemSaldo

Transferir A->B

Depositar AINDA NAO IMPLEMENTADO.
Escreva o teste, veja o vermelho, e so entao o corpo.

O que ja existe
Este painel le os nove metodos e diz quais ja tem corpo. Ele e o placar da sua noite.

[] Somar

[] Subtrair

[] Multiplicar

[] Dividir

[] Porcentagem

[] Depositar

[] Sacar

[] TemSaldo

[] Transferir

Não é bug.

É um contrato assinado e não cumprido. O método Somar existe — tem nome, recebe dois números e promete devolver um terceiro. É por isso que a janela abre.

Os nove respondem igual

Nenhum dos nove métodos tem corpo. A janela não esconde isso: ela diz, em cada clique, que a conta ainda não foi escrita.

UC11 · Aula 7 · 09/09/2026

O painel da direita é o seu placar

A janela lê os nove métodos e diz quais já têm corpo. À esquerda, como ela chega na sua mão. À direita, como ela vai estar quando você terminar.

O que já existe

Este painel lê os nove métodos e diz quais já tem corpo. Ele é o placar da sua noite.

- ☐ Somar
- ☐ Subtrair
- ☐ Multiplicar
- ☐ Dividir
- ☐ Porcentagem
- ☐ Depositar
- ☐ Sacar
- ☐ TemSaldo
- ☐ Transferir

O que já existe

Este painel lê os nove métodos e diz quais já tem corpo. Ele é o placar da sua noite.

- ☒ Somar
- ☒ Subtrair
- ☒ Multiplicar
- ☒ Dividir
- ☒ Porcentagem
- ☒ Depositar
- ☒ Sacar
- ☒ TemSaldo
- ☒ Transferir

0 de 9 -> 9 de 9

O título da janela conta os métodos implementados. Ele começa em zero e sobe a cada corpo que você escreve.



É por isso que a janela existe

Você vê o efeito do seu código sem precisar ler a barra de testes. Mas repare na ORDEM: o [X] acende DEPOIS do teste vermelho, nunca antes. Quem implementa para ver o verde acender inverteu a noite.

Assinatura e corpo

Todo método tem duas partes. A de cima é a promessa; a de baixo é o cumprimento.

Calculadora.cs — como ele chega na sua mão

```
1 public static double Dividir(double a, double b)
2 {
3     throw new NotImplementedException("Escreva o teste primeiro.");
4 }
```

A ASSINATURA

Nome, parâmetros e tipo de retorno. Ela JÁ ESTÁ ESCRITA em todos os nove métodos.

O CORPO

A conta. Está vazio, e no lugar dele há um throw que derruba quem chamar.

Por que as assinaturas vêm prontas

Você poderia perguntar por que o professor não entregou o arquivo em branco. A resposta é técnica, e vale entender: um teste que chama um método inexistente não fica vermelho — ele impede o projeto inteiro de compilar.

```
SEM ASSINATURAo projeto NAO COMPILA, e voce fica sem barra nenhuma  
COM ASSINATURAo projeto compila, e a barra fica VERMELHA  
A DIFERENCA    vermelho e informacao; nao compilar e so silencio
```



A regra prática

Escrever um teste não cria método nenhum. Se você inventar um nome que não está no arquivo, nada compila — e aí ninguém na sala consegue rodar coisa alguma.

P A R T E

2

A regra vira teste

- REGRA, CASO e TESTE
- O critério de aceitação
- Como se dá nome a um teste

Três linhas, sempre nesta ordem

A folha impressa que você recebeu se preenche ANTES de abrir o Visual Studio. Cada regra do sistema ocupa três linhas:

```
REGRA 3:  Nao da para sacar mais do que tem na conta.
CASO:     conta com 100, sacar 150  ->  recusa, e o saldo continua 100
TESTE:    Sacar_ComValorMaiorQueSaldo_RecusaEMantemSaldo
```

A linha	O que ela é	De onde vem
REGRA	O que o sistema tem de fazer, em português	Da sua cabeça, e do quadro
CASO	Um exemplo com número, e o resultado esperado	Você escolhe os valores
TESTE	O nome do método de teste	Metodo_Cenario_OQueDeveDar

Uma regra sem caso não vira teste

ISTO NÃO É REGRA

"A senha tem de ser forte."

Ninguém consegue escrever um Assert a partir disso. Forte quanto?
Medido como?

ISTO É REGRA

"Senha com menos de 8 caracteres é recusada."

Dá para escolher um valor, chamar o método e comparar.

A diferença entre as duas é o CRITÉRIO DE ACEITAÇÃO: dizer, antes de testar, o que "estar certo" significa para aquele método. Sem ele não existe teste — existe opinião.



Por que isso é trabalho seu, e não do programador

Quem escreve o código já tem uma resposta na cabeça, e vai testar aquilo que ele mesmo imaginou. Decidir o critério ANTES, e por escrito, é o que impede o programa de definir sozinho o que é correto.

O nome do teste conta a regra

O nome não é enfeite: é ele que aparece na barra quando o caso fica vermelho. Um nome bom dispensa abrir o arquivo para entender o que quebrou.

```
Metodo_Cenario_OQueDeveDar
```

```
NaGarantia_ComNoventaDias_DevolveVerdadeiro
```

```
NaGarantia_ComNoventaEUmDias_DevolveFalso
```

```
Sacar_ComValorMaiorQueSaldo_RecusaEMantemSaldo
```

NUNCA

teste1, teste2, testeSomar, TesteDeSoma

SEMPRE

o método, a situação, e o que tem de acontecer

P A R T E

3

0 ciclo: vermelho primeiro

- Os quatro passos, e por que a ordem é rígida
- O exemplo completo, do zero ao verde
- Dois vermelhos diferentes

A ordem da noite

1. REGRA 2. TESTE 3. VERMELHO 4. CODIGO 5. VERDE

O passo 3 é o que mais se pula, e é justamente o que não pode faltar. Ele existe para provar que o teste está ligado no código — que ele realmente chama o método e realmente olha o resultado.

verde em qualquer programa do mundo, e nao prova nada

```
1 [TestMethod]
2 public void EsteTestePassaSempre()
3 {
4     Assert.IsTrue(true);
5 }
```

 **Um teste que nasce verde é suspeito**
Você nunca o viu falhar, então não sabe se ele está preso ao código ou testando o nada. Ver o vermelho uma vez é o que compra essa confiança.

0 exemplo completo — a garantia

Um sistema de assistência técnica. A regra, ditada pelo cliente: o produto está na garantia até o 90.o dia, INCLUSIVE. No 91.o já não está.

passo 1 — a regra, no papel

```
REGRA:  na garantia ate o 90o dia, inclusive
CASO:   90 dias -> verdadeiro | 91 dias -> falso
TESTE:  NaGarantia_ComNoventaDias_DevolveVerdadeiro
```

passo 2 — o teste, antes do código

```
1 [TestMethod]
2 public void NaGarantia_ComNoventaDias_DevolveVerda
3 deiro()
4 {
5     Assert.IsTrue(Garantia.NaGarantia(90));
6 }
```

passo 3 — o vermelho que se espera

```
1 Com falha  NaGarantia_ComNoventaDias
2
3 Test method threw exception:
4 System.NotImplementedException
```


Dois vermelhos, e eles dizem coisas diferentes

o metodo nao existe ainda

```
1 Test method threw exception:  
2 System.NotImplementedException:  
3 Escreva o teste primeiro.
```

o metodo existe, e respondeu errado

```
1 Assert.AreEqual failed.  
2 Expected: <90>  
3 Actual:   <89>
```

O da esquerda é o vermelho do começo: ninguém escreveu o corpo, e o throw derrubou a chamada. O da direita é o vermelho de verdade: o código rodou até o fim e devolveu um valor que não bate com a regra.



Hoje você vai ver os dois, nesta ordem

Primeiro o da esquerda, em todo método. Depois de implementar, se aparecer o da direita, é porque o seu código e a sua regra discordam — e um dos dois está errado.

O kit de Assert desta noite

O que se escreve	O que ele cobra
<code>Assert.IsTrue(x)</code>	x tem de ser verdadeiro
<code>Assert.IsFalse(x)</code>	x tem de ser falso
<code>Assert.AreEqual(esperado, obtido)</code>	os dois têm de ser iguais — o ESPERADO vem primeiro
<code>Assert.AreEqual(esp, obt, FOLGA)</code>	iguais com folga — obrigatório para double
<code>Assert.ThrowsException<T>(...)</code>	o que se espera não é um valor: é um estouro



O esperado vem primeiro

No AreEqual, o primeiro valor é o que a REGRA manda e o segundo é o que o programa devolveu. Trocar a ordem não muda a cor da barra, mas inverte a mensagem de erro — e aí você lê "esperado 89, obtido 90" quando é o contrário.

Quando o esperado é um estouro

Nem toda regra termina em valor. Às vezes a resposta certa é o programa recusar a pergunta — e aí não há nada para comparar com `AreEqual`.

o teste passa SE o metodo estourar

```
1 // a regra diz que prazo negativo e ERRO, e nao uma resposta
2 [TestMethod]
3 public void NaGarantia_ComPrazoNegativo_Estoura()
4 {
5     Assert.ThrowsException<ArgumentException>(
6         () => Garantia.NaGarantia(-5));
7 }
```



Guarde este molde

Você pode precisar dele hoje: em pelo menos um método a turma pode decidir que a resposta certa não é um número, e sim um erro. Qual método é esse, vocês decidem no quadro.

P A R T E

4

Muitos casos, um teste só

- O problema de escrever um método por caso
- [DataTestMethod] e [DataRow]
- O erro que não compila

Seis casos, seis métodos?

A sua folha tem várias regras para o mesmo método. Escrever um [TestMethod] para cada uma funciona — e vira um arquivo enorme, com o mesmo Assert copiado seis vezes.

e ainda faltam quatro

```
1 [TestMethod]
2 public void NaGarantia_Com89_DevolveVerdadeiro()
3 {
4     Assert.IsTrue(Garantia.NaGarantia(89));
5 }
6
7 [TestMethod]
8 public void NaGarantia_Com90_DevolveVerdadeiro()
9 {
10     Assert.IsTrue(Garantia.NaGarantia(90));
11 }
```



O que muda de um para o outro

Só o número. O nome muda junto, mas a lógica é a mesma linha repetida — e linha repetida é lugar onde erro se esconde.

A tabela de casos

quatro casos, um metodo de teste

```
1 [TestMethod]
2 [DataRow(0, true, "comprado hoje")]
3 [DataRow(89, true, "vespera do limite")]
4 [DataRow(90, true, "LIMITE: o ultimo dia, inclusive")]
5 [DataRow(91, false, "um dia depois")]
6 public void NaGarantia_PorTabela(int dias, bool esperado, string caso)
7 {
8     Assert.AreEqual(esperado, Garantia.NaGarantia(dias), caso);
9 }
```

[TestMethod]

Substitui o [TestMethod]. Sem ele os [DataRow] são ignorados em silêncio.

Os valores

A ordem dentro do [DataRow] é a mesma ordem dos parâmetros da assinatura.

O último texto

É o rótulo. Aparece na barra e diz qual caso caiu.

O erro que não compila

É o engano mais comum com [DataRow], e ele não aparece como vermelho na barra: aparece como projeto que não compila. A regra é uma só, e é de contar.

A REGRA	cada [DataRow] tem de ter o MESMO numero de valores
QUE	a assinatura tem de parametros
ENTAO	3 valores no DataRow -> 3 parametros no metodo

```
[DataRow(90, true, "o limite")]           // 3 valores
public void X(int dias, bool esperado)     // 2 parametros -> NAO COMPILA

[DataRow(90, true, "o limite")]           // 3 valores
public void X(int dias, bool esperado, string caso) // 3 -> ok
```

 **Acrescentou o rótulo depois?**
Então acrescente o parâmetro também, em todos os DataRow daquele método. É por isso que vale escolher o formato antes de encher a tabela.

PARTE

5

0 número que não é igual a ele mesmo

- Por que $0.1 + 0.2$ não dá 0.3
- A FOLGA, e por que ela não é preguiça

0.1 + 0.2 não dá 0.3

```
double a = 0.1 + 0.2;

// a vale 0.30000000000000004

Assert.AreEqual(0.3, a);           // VERMELHO, e o código está certo
Assert.AreEqual(0.3, a, 0.0001);  // verde
```

O computador guarda número com casa decimal em base 2, e 0.1 não tem representação exata em base 2 — do mesmo jeito que um terço não tem representação exata em base 10. A conta erra numa casa lá no fim, sempre.



A FOLGA não é preguiça

Ela é o terceiro parâmetro do `AreEqual`, e diz o quanto de diferença ainda conta como igual. Nos seus arquivos de teste ela já vem declarada no topo: `FOLGA = 0.0001`. Use, e não a apague.

PARTE

6

Quando o objeto guarda coisa

- A conta tem saldo, e o saldo muda
- O [TestInitialize]

Um teste não pode enxergar o outro

A Calculadora recebe dois números e devolve um terceiro, sem lembrar de nada. A Conta é diferente: ela tem saldo, e cada depósito muda o saldo. Isso cria um problema novo.

```
// se os dois testes usarem a MESMA conta:  
  
teste A:  deposita 100  ->  saldo 100    (passa)  
teste B:  deposita  50  ->  saldo 150    (esperava 50)  
  
// e o teste B passa ou falha dependendo da ORDEM
```



É o pior tipo de vermelho

Ele aparece e some conforme a ordem em que os testes rodam, e manda você caçar um defeito que não existe no código de produção — está no seu teste.

0 [TestInitialize]

Um método marcado com [TestInitialize] roda ANTES DE CADA teste da classe. Não é você que o chama — o adaptador chama, sozinho, toda vez.

ja vem pronto no seu ContaTests.cs

```
1 private Conta conta;  
2 private Conta destino;  
3  
4 [TestInitialize]  
5 public void AntesDeCadaTeste()  
6 {  
7     conta = new Conta();  
8     destino = new Conta();  
9 }
```



Todo teste começa do zero

Com ele, cada teste ganha duas contas novas e zeradas, e nenhum enxerga o que o outro fez. A ordem deixa de importar — que é exatamente o que teste unitário precisa.

PARTE

7

O contrato de hoje

- Nove métodos, nenhum com corpo
- As perguntas que a turma tem de responder

Os nove métodos

Calculadora	
Somar(a, b)	devolve a soma
Subtrair(a, b)	o primeiro menos o segundo
Multiplicar(a, b)	o produto
Dividir(a, b)	o primeiro dividido pelo segundo
Porcentagem(valor, pct)	quanto é o percentual do valor

Conta	
Saldo	quanto tem — só leitura
Depositar(valor)	põe dinheiro
Sacar(valor)	tira, e diz se deu certo
TemSaldo(valor)	só pergunta, não gasta
Transferir(destino, valor)	tira de uma, põe na outra



O que está escrito acima é tudo o que existe

Nenhum desses métodos tem corpo. As descrições da direita são o resumo da REGRA que está no comentário de cada um — e o comentário é o mais longe que o material vai. O resto sai da sua folha.

As perguntas sem resposta impressa

Estas não estão respondidas em lugar nenhum — nem no código, nem nesta apostila. A turma decide junto, no quadro, e a decisão vira regra para todo mundo.

A situação	A pergunta
Dividir(10, 0)	erro? um número? qual?
Multiplicar(3, -4)	o sinal do resultado
Depositar(-30)	aceita, ignora ou recusa?
Sacar(100) com saldo 100	o limite é para dentro ou para fora?
Sacar(-50)	e se tirar um negativo?
Transferir para a própria conta	vale?



Depois de decidida, ela vale

Quem implementar diferente do que a turma combinou vai ficar vermelho — e vai estar errado, mesmo que o código funcione. É assim que funciona num projeto de verdade.

PARTE

8

0 que a barra verde não prova

- O teste que você não escreveu
- O fim da noite

A barra verde prova pouco

No fim da noite a sua suíte vai estar verde. Antes de comemorar, volte à garantia — a suíte abaixo está inteira verde:

3 casos, 3 verdes, e nenhuma prova

```
[TestMethod]
[DataRow(10, true, "dentro do prazo")]
[DataRow(30, true, "dentro do prazo")]
[DataRow(200, false, "fora do prazo")]
public void NaGarantia_PorTabela(int dias, bool esperado, string caso)
```

Nenhum dos três casos chega perto do 90. Um programa que recusasse a partir do dia 50 passaria nos três — e a barra diria verde do mesmo jeito. A suíte não está errada: ela está incompleta, e as duas coisas se parecem muito de longe.

O teste que voce nao escreveu
e o defeito que voce vai entregar.

Ao sair hoje

Você escreveu testes para um código que não existia, e depois escreveu o código. Essa ordem tem nome no mercado, e é ela que faz a diferença: o teste deixa de ser conferência e vira especificação.

Volte à folha de regras e responda as três perguntas do fim. A terceira — quantas regras apareceram só quando o professor rodou a versão sabotada — é a mais importante, e ela não vale nota nenhuma. Todo mundo tem um número ali, inclusive quem programa há vinte anos.

Amanhã o assunto é o que acontece quando você conserta uma coisa e quebra outra.